CR-PROJET-SEMESTRE

APELOIG, CHERITI, DESCHAMPS, STRAPPAZZON, TROUMELIN

December 2022

1 Introduction :

Vous trouverez dans ce document un résumé des processus mis en oeuvre pour la réalisation de notre logiciel destiné à la borne de location vidéo AL2000. Nous avons fait de notre mieux pour répondre aux attentes du SRS de Mr.Jouanot. Le travail que nous vous présentons est certes incomplet au 12 décembre 2022, mais il offre une vue d'ensemble correcte des solutions que nous avons implémentées.

2 Organisation et ressources:

Nous sommes 5 étudiants en master 1 d'informatique classique. Lors d'un cours de Génie Logiciel, nous avons reçu les attentes d'un client, CyberVideo. À l'aide de celles-ci, nous avons réalisé un SRS (Software Requirements Spécification) que vous pourrez trouver joint à ce rapport. Simultanément, nous avons réalisé un ensemble de schéma de conception (UMLs, schéma d'architecture de base de données, etc...) que nous avons rendu dans un temps intermédiaire. Le travail fut ensuite réparti en trois pôles que nous expliciterons ci-dessous : Base de données, noyau fonctionnel et Interface Humain-Machine.

3 BD

3.1 Initialisation

Pour exécuter le projet, vous aurez besoin des .jar externes suivants (disponibles à l'adresse entre parenthèses) :
- jsch-0.1.54.jar (https://mvnrepository.com/artifact/com.jcraft/jsch/0.1.54)
- json-simple-1.1.1.jar (http://www.java2s.com/Code/Jar/j/Downloadjsonsimple111jar.htm)
- mybatis-3.4.1.jar (https://mvnrepository.com/artifact/org.mybatis/mybatis/3.4.1)
- ojdbc14.jar (http://www.java2s.com/Code/Jar/o/Downloadojdbc14jar.htm)

Pour démarrer le projet, il nous fallait une base de données complète, afin de faciliter les tests. Sur suggestion de Mr.Jouanot, nous nous sommes penchés sur l'API du site The Movie Database (disponible à l'adresse suivante : https://www.themoviedb.org/), et nous l'avons utilisée pour remplir notre propre base de données.
Nous avons décidé de réaliser nos tests sur une base de données en locale grâce à OracleExpress et SQLDeveloper, pour éviter les complications liées à celle de l'IM2AG (connexions compliquées,

droits limités, ...).
Pour automatiser le processus et récupérer un millier de film, nous avons suivi ces étapes :
1. Grâce à une clef d'API fournie par le site, nous avons pu réaliser des requêtes directement en https pour nous renvoyer sur les pages disposant des informations sur un film. Pour récupérer toutes les informations intéressantes, il faut réaliser 4 requêtes par film : une pour l'image, une pour les genres, une pour les crédits, et une pour le titre.
La forme d'une requête pour les titres :

```
https://api.themoviedb.org/3/movie/" + i + "? " + api_key + "+ "&language=fr-FR
```

Une fois toutes les requêtes pré-créées et stockées dans un fichier texte (par type de requête), elles sont appelées quatre à quatre pour aller sur les pages concernées.
2. Grâce au downloader développé en TLI avec Mr.Blanch, on télécharge chaque page, et nous les stockons dans des dossiers séparés par type (1 par type de requête). Nous recevons à ce moment-là des fichiers JSON, il a fallu utiliser un outil de traductions, ou bien faire du regex et les considérer comme des Strings. Nous avons choisi la première option. Nous avons donc créé une classe traduction depuis les JSON dans les formats qui nous arrangeaient.
3. Grâce à la traduction, nous avons pu fusionner dans des tableaux ou des maps les informations nécessaires à chaque film.
4. Enfin, il a fallu initialiser la base de données avec les informations recueillies à l'aide des packages java dédiés au SQL. Nous avons donc utilisé dans un premier temps un ScriptRunner pour lire et exécuter un script construisant l'architecture de la base de données. Puis des PreparedStatement afin d'insérer les films.
Initialement notre BDD devait être d'environ 1000 films, mais seulement 98 ont étaient réelemment insérés dans la BDD de l'IM2AG, en raison du nombre de curseurs ouverts autorisés limités. En relançant le programme (en enlevant à la main les fichiers déjà insérés dans la BD), il est cependant possible de relancer autant de fois que nécessaire, afin d'obtenir la base de données à laquelle nous aspirions.

3.2 Gestion de l'accès aux données

L'accès aux données se fait à l'aide d'un package DAO qui contient les classes nécessaires aux interactions entre la BDD et notre programme. Il distingue deux cas : le premier se fait avec une base de données Oracle local que nous avons utilisée pour la plupart des tests. Le deuxième est le cas où la base de données est distante sur le serveur Oracle mis a disposition par la fac dans notre cas. Dans le package DAO, les connexions et requêtes envoyées sont gérées via les classes DaoConnection et DaoExecQuerry. La classe DaoConnection étant un singleton, on s'assure qu'il n'y ait qu'une seule instance de la connexion à la base de données.

Fonctionnement de DAOConnection :
La classe DaoConnection initialise la connexion au serveur Oracle en passant par un tunnel ssh qui fait le lien entre le serveur Mandelbrot et le serveur Oracle. La connexion en SQLplus se fait à l'aide d'un pilote jdbc :

```
DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver());
```

Elle fournit alors les méthodes connectDB() et disconnectDB pour s'interfacer avec la BDD.

Fonctionnement de DaoExecQuery :
La classe DaoExecQuery sert à envoyer une requête SQL à la BDD. Elle prend comme paramètre une requête SQL sous forme de chaîne de caractères et retourne le résultat de cette requête sous la forme d'un ResultSet. Nous avons décidé d'uniformiser les retours de requête en traitant resultSet et en le transformant en un ArrayList d'HashMap. Un format qui représente la table de retour. Chaque entré dans l'array list correspond à une ligne de la table. Pour chaque ligne représentée par une HashMap qui a pour clé le nom des colonnes et pour valeur, la valeur des colonnes pour la ligne courante. C'est ce format des tables qui est retourné lorsqu'il y a une requête du noyau fonctionnel.

3.3 Architecture

Concernant l'architecture de la base de données, nous avons conçu le schéma disponible en annexe, Figure 1.
Un AL2000 a un identifiant, une localisation, et un stock de films physiques. Ce stock est constitué de 100 films physiques qui possèdent un identifiant, un état (RENTABLE, RENTED, DAMAGED, UNVERIFIED), et une référence à sa version universelle, i.e., celle stockée, avec les détails, dans LesFilms. Chaque film possède un identifiant unique, qui sert à récupérer tous ses détails : titre, réalisateur, acteurs, sa description, son année de sortie, l'URL de son affiche ou une URL renvoyant à une ImageNotFound dans le cas où l'affiche n'est pas disponible, et ses genres. Nous avions au début une table genre, mais pour des raisons de facilités et parce qu'un film n'a jamais plus de 7 genres, nous avons intégré ceux-ci directement dans leur table. À chaque location, la table correspondant à la méthode de location est mise à jour, et la location est rattachée à l'abonné. Un client peut avoir plusieurs abonnements (représentés ici par plusieurs abonnés) et on garde une trace des crédits pour chacun de ces abonnements. Enfin, les techniciens et les administrateurs sont deux acteurs ayant des droits et méthodes différentes.

3.4 Triggers et contraintes

Les contraintes de la base de données en elle-même sont disponibles dans le script SQL (Annexe **??**) ainsi que le code des triggers. Nous avons implémenté ceux-ci :

```
    - Trigger lotofdamaged : Au rendu d'un film physique, si ce film est
    "DAMAGED" ou "UNVERIFIED", un technicien est appelé si le nombre total de
    films physiques dans l'un ou l'autre de ces états dépasse 10.

    create or replace TRIGGER lotofdamaged
    AFTER UPDATE ON FilmPhysique
        DECLARE NB NUMBER;
    BEGIN
        SELECT COUNT(*) INTO NB FROM FilmPhysique, StockFilmsPhysiques
    WHERE (ETAT='DAMAGED' OR ETAT='UNVERIFIED') AND FilmPhysique.ide=
StockFilmsPhysiques.ide_film_physique;
    IF(NB>10) THEN
     raise_application_error(-20111,'Appel technicien');
    END IF;
END;
```

- Trigger OodCreditHisto : Quand un abonné rend un film, tous les crédits de plus d'un an sont supprimés de la table historiqueCredit.

```
create or replace TRIGGER OodCreditHisto
AFTER INSERT ON HistoriqueLocationPhysique
FOR EACH ROW
BEGIN
    DELETE FROM HistoriqueCredit
    WHERE date_credit < SYSDATE - INTERVAL '1' YEAR;
END;
```

- Trigger refunddamaged : au passage d'un technicien, si celui-ci confirme l'état délétère d'un BluRay (en faisant passer son état de UNVERIFIED à DAMAGED), le compte de l'abonné est crédité de la somme qu'il avait mis dans la location du BluRay.

```
create or replace TRIGGER refunddamaged
AFTER UPDATE ON FilmPhysique
FOR EACH ROW
DECLARE
    searchid INTEGER;
    montant INTEGER;
BEGIN
    SELECT id_abonne INTO searchid FROM HistoriqueLocationPhysique WHERE
    id_film_physique=:NEW.ide AND dfloc= (select max(datedate) from
    (SELECT ddloc as datedate FROM HistoriqueLocationPhysique WHERE
    id_film_physique=:NEW.ide));
    SELECT price INTO montant FROM HistoriqueLocationPhysique
    WHERE id_abonne=searchid AND id_film_physique=:NEW.ide;
    IF (:NEW.etat='DAMAGED') THEN
        UPDATE Abonne
        SET montant_carte_abonnement = montant_carte_abonnement + montant
        WHERE ide = searchid;
    END IF;
END;
```

- Trigger DevotedClient : à la location d'un film, si c'est le 20ème film que l'abonné loue ce mois-ci, son compte est crédité de 10 euros.

- Trigger CalmRequest : À la requête d'un film en version physique pour l'AL2000, si une demande pour le même film existe déjà, la requête est ignorée. De même si une version physique existe déjà dans l'AL2000.

3.5 Implémentation

2 diagrammes UML sont disponibles dans le dossier de rendu. La partie DAO gère les requêtes et notifications envoyées depuis le noyau fonctionnel. Les requêtes sont reçues par un médiateur qui est notre classe DaoMediatorConcret. Ce médiateur implémente donc les méthodes qui vont appeler les différentes classes nécessaires à la gestion de ces requêtes, et permet de faire des retours au noyau fonctionnel.

Les notifications envoyées depuis le noyau fonctionnel ne nécessitent pas de retour de la DAO, elles servent donc à mettre à jour la BD en fonction des différents événements reçus depuis le noyau fonctionnel. Nous avons implémenté un observateur qui permet de gérer ces événements. C'est la classe DaoEventManager qui s'occupe de lier les bonnes classes aux bons évènements. Une fois que toutes les classes sont abonnées à leurs événements, DaoEventManager notifie celles concernées à chaque fois qu'un événement est envoyé. Chaque classe va alors exécuter sa propre méthode et aller chercher la bonne requête. celle-ci sera construite en fonction des informations liées a cet événement, grâce à la classe QueryBuilder qui s'occupe de générer toutes les requêtes SQL.

3.6 Limites, état actuel

En l'état actuel des choses, la partie DAO a plusieurs failles. Tout d'abord, deux de nos triggers ne sont pas implémentés par manque de temps. Ensuite, bien que fonctionnant sur la base de donnée locale que nous avons initialisée, notre code est très peu portatif : La partie d'initialisation de notre code est monolithique, "programmation fonctionnelle", et demande pour cela des ajustements faits à la main pour de nouveaux portages. Cela est dû à plusieurs facteurs, notamment du fait de la nécessité des fichiers téléchargés comme expliqué plus tôt, et que l'implémentation de cette partie soit destinée uniquement à un usage personnel. La meilleure pratique aurait été de se servir de cette partie pour remplir un fichier sql que nous aurions exécuté après l'initialisation de la l'architecture de la base en elle-même. Dès lors, il aurait fallu connaître, où en tout cas c'est ce que nous croyons, la configuration de la base cible (notamment au niveau du nombre de curseurs ouverts autorisés). Cependant, avec un petit peu de refactoring, nous pensons qu'il ne serait pas trop difficile de faire disparaître les classes responsables du scrapping de l'API, et de fournir un script correct, si tant est que l'on se limite sur le nombre de requête. Notons tout de même que, dans son implémentation actuelle, la base de données est riche et fonctionne en locale de manière efficace.

4 Noyau fonctionnel :

4.1 Début de la conception et TDD

Dans un premier temps, nous avons rendu un ensemble de diagrammes de classes, et, après les retours et discussion avec notre enseignant de conception orientée objet, nous avons réalisé que cette première approche était une approche très naïve de la conception objet. Nous sommes donc repartis sur une nouvelle approche pour développer notre logiciel : le Test Driven Development.
Cette technique de développement nous paraissait intéressante car nous n'avions jamais essayé de développer une solution en fonction du code de comportements déjà écrit (à part lorsque les enseignants nous les fournissaient (comme pour l'allocateur mémoire développé en C.S.E.-P.C.), cette

technique nous a permis de mettre en lumière au bout de quelques sessions que notre compréhension de la conception orientée objet était très naïve (nous avions fait des objets à partir d'entités réelles de manière très naïve), nous avons donc décidé de continuer à développer les simples comportements calculatoires du noyau fonctionnel de cette manière, puis lorsque ce fut fini, nous avons récupéré ces méthodes plus tard dans la nouvelle conception que nous expliquons ci-dessous.

4.2 Conception orientée objet et SOLID :

Malheureusement du fait de notre manque de maitrise de cette approche et la piètre qualité de notre UML de classes, la TDD nous a surtout permis de mettre en lumière les limitations de celui-ci. Nous sommes donc entré dans une nouvelle phase de conception et avons repris le design de la partie coeur fonctionnel et son interaction avec les autres (IHM et DAO) en gardant à l'esprit la notion de SOLID (Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, Dependancy Inversion Principle). Dans cet esprit nous avons essayé de limiter au maximum l'existence d'une "GOD" classe du type AL2000 portants la responsabilité de l'ensemble des états interne du système et nous fournissons à l'IHM et à la BDD un ensemble d'interfaces que nous allons décrire ci-dessous. Dans un premier temps nous avons l'**UiObserver.class** qui est une spécialisation de l'interface *Observer*, cette classe garantis à n'importe quel UI qui a son interaction écoutée par elle un comportement fixé, les éléments du noyau fonctionnel implémentant *ComponentFC* peuvent recevoir des updates sous la forme :

```
public void update(String EVENT_TYPE, ArrayList<String> data);
```

Suite à un message de l'UI qui prends la forme :

```
observer.notify(String PAGE_EVENT_TYPE, ArrayList<String> data);
```

Après réflexion, mais une fois l'implementation trop avancée nous aurions du implementer un type Data pour les données transmises dans un format établi par nous. À l'heure actuelle, l'utilisateur (les devs de l'ihm) doivent respecter des formats spécifiques pour les **ArrayList**$<$ *String* $>$ **data** qui dépendent des **EVENT_TYPE**, nous aurions donc du encapsuler cela pour rendre l'utilisation des signaux plus modulaires. De même les EVENT_TYPE devraient être des enum, à l'instant nous fournissons les règles de l'observer suivante à l'UI :

```
data is an ArrayList<String>
        fc.Machine
RENT_BLURAY_EVENT_TYPE , data as : String[0] wich is the BluRayId
RENT_QRCODE_EVENT_TYPE , data as :
RETURN_DAMAGED_EVENT_TYPE , data as : String[0] BluRay id
RETURN_CORRECT_EVENT_TYPE , data as : String[0] BluRay id
GUEST_IN_EVENT_TYPE , data as : String[0] wich is the credit card
number
SIGN_IN_EVENT_TYPE , data as :  String[0] wich is the subscriber
number
SIGN_UP_EVENT_TYPE , data as :String[0]: Bank CardNumber
                              String[1]: Subscriber Name
                              String[2]: Password
                              String[3]: Adress
```

```
                              String[4]: Blocked Genre
                              String[5]: Initial Balance
SIGN_OUT_EVENT_TYPE , data as : String[0] User id
RENT_PRINTQRCODE_EVENT_TYPE , data as String[0] rented movie id
RENT_FREEBLURAY_EVENT_TYPE , data as String[0] rented movie id

fc.search
SEARCH_BY_TAG , data as String[n] = keyword n
SEARCH_NO_TAG , data as String[n] = keyword n

fc.User
UPDATE_SUBSCRIBER_EVENT_TYPE , data as :String[0]: Subscriber Name
                                        String[1]: Password
                                        String[2]: Adress
                                        String[3]: Blocked Genre
                                        String[4]: Current Balance
AUTHENTIFICATION_EVENT_TYPE , data as : String [0] User password
RENT_BLURAY_EVENT_TYPE , data as : String[0] BluRay id
RETURN_DAMAGED_EVENT_TYPE , data as : String[0] BluRay id
RETURN_CORRECT_EVENT_TYPE , data as : String[0] BluRay id
QRCODE_PAIE_EVENT_TYPE , data as : String[0] BluRay id
                                   String[1] User id
REFOUND_TECHNICAL_EVENT_TYPE , data as : String[0] UserId to refund
```

Pour les affichages de la **GUI** nous fournissons les modèles du **FC** nécessaires à l'affichage d'information (dont nous détaillons les interfaces en annexes) pour l'utilisateur actuel, le moteur de recherche, les résultats d'une recherche, l'historique d'un utilisateur.
Pour le design interne du **FC** vous pourrez trouver ci-joint un uml de classe de notre noyau fonctionnel nous allons détailler ci-dessous les quelques décisions de design que nous avons pris qui sont sans rapport avec la GUI :

- La classe **Machine.class** fait office de médiateur pour le noyau fonctionnel, nous l'utilisons également pour initialiser le "logiciel" comme si nous appuyons sur le bouton on d'un AL2000, de fait elle a également les méthodes pour "imprimer" les QrCode et "sortir" les Blu-Ray

- Nous avons décidé d'utiliser le design pattern **Singleton** pour un certain nombre de classes qui nous semblaient pertinentes : **Machine.class** , **BasicSearch.class**, **DAORequestHandler**, **BluRayStock** ces classes n'ayant pas de rapport avec les utilisateurs pour des raisons de performance s'assurer qu'elles soient uniques nous paraissaient pertinent.

- Abstraction de la majorité de nos classes pour fournir des interfaces aux divers médiateurs et observateurs.

- La majorité de nos classes passent par des builders pour pouvoir sortir la construction des classes elles-mêmes.

- La classe *DAORequestHandler* est une classe abstraite ce qui nous a permis de réaliser un **DAORequestHandlerExample.class** pour pouvoir faire des tests sans avoir à lier la DAO au noyau fonctionnel.

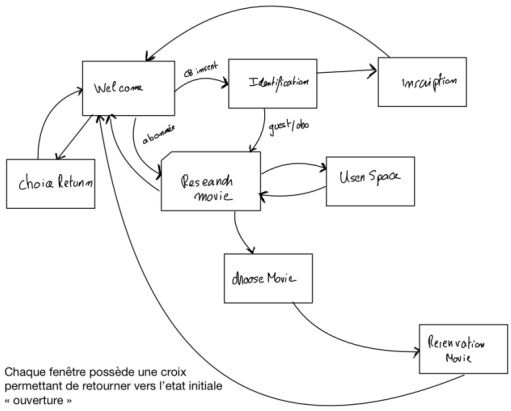
- la classe **ClickeableTag** est un décorateur de **Tag.class** qui nous permet d'ajouter le comportement cliquable aux tags ce qui est utile dans notre modele pour le moteur de recherche.

4.3 Tests :

Initialement lorsque nous faisions l'implementation de notre code en Test Driven Developement, nous avons pu faire des tests unitaires des comportements calculatoire du noyau fonctionnel qui sont trouvables sur les branches **tdd_** de notre git [1], une fois le changement de paradigme de développement acté et du fait de notre manque d'organisation nous avons décidé de re-implementer les aspects calculatoires déjà testé puis de faire des tests integrés avec la **GUI**.

5 Interface Humain-Machine:

Cette dernière partie correspond à l'aspect visuel de l'AL2000. Dans un premier temps nous avons réalisé un ensemble de fenêtres graphiques indépendantes que nous avons pu tester individuellement [2]. Puis nous avons mis en place une machine à état qui permet de faire la transition entre les pages, de maintenir des modèles des données du noyau fonctionnel c'est également elle qui prend l'**UIObserver** en attribut pour pouvoir lui envoyer les messages nécessaires pour le fonctionnement du FC, de plus nous avons mis en place une série de **GUI_EVENT** interne qui permet aux pages d'effectuer leurs changements et transition au sein de là l'interface visuelle. Vous trouverez ci-dessous un diagramme de la machine à état de l'UI.



6 Bilan :

Le bilan de ce projet est positif à nos yeux, même si nous n'avons pu remplir les attentes du SRS. En effet, les membres du groupe de ce projet ont pu acquérir des compétences significatives dans les divers domaines étudiés cette année. Nous avons pu notamment commencer à integrer les notions de **SOLID** dans le design de systèmes et d'IHM, et nous avons pu travailler sur les triggers en base de données. Néanmoins, nous restons néophytes en la matière et continuerons de nous exercer dans

[1] https://github.com/Maleusa/al2000_software
[2] Nos talents de designer furent mis à rude épreuve, le résultat visuel est à la hauteur de ceux ci.

ces domaines.
Il nous paraît également clair que nous avons manqué d'organisation durant ce projet. Une fois la phase de génie logiciel passée et les tâches distribuées, nous nous sommes grandement relâché sur le maintien de la cohérence du projet, chaque pôle travaillant dans son coin. Nous devrons donc pour un prochain de cette envergure établir un échéancier et faire a minima des réunions régulières entre tous les membres du groupe.
Pour la suite de ce projet en particulier, nous comptons lier la base de données au logiciel pour pouvoir faire des tests de plus grande envergure, si d'autres souhaitent le reprendre, les choses nous paraissant les plus intéressantes à travailler sont l'encapsulation des données transmises par les observateurs et le design/ergonomie de l'IHM.

7 Annexes :

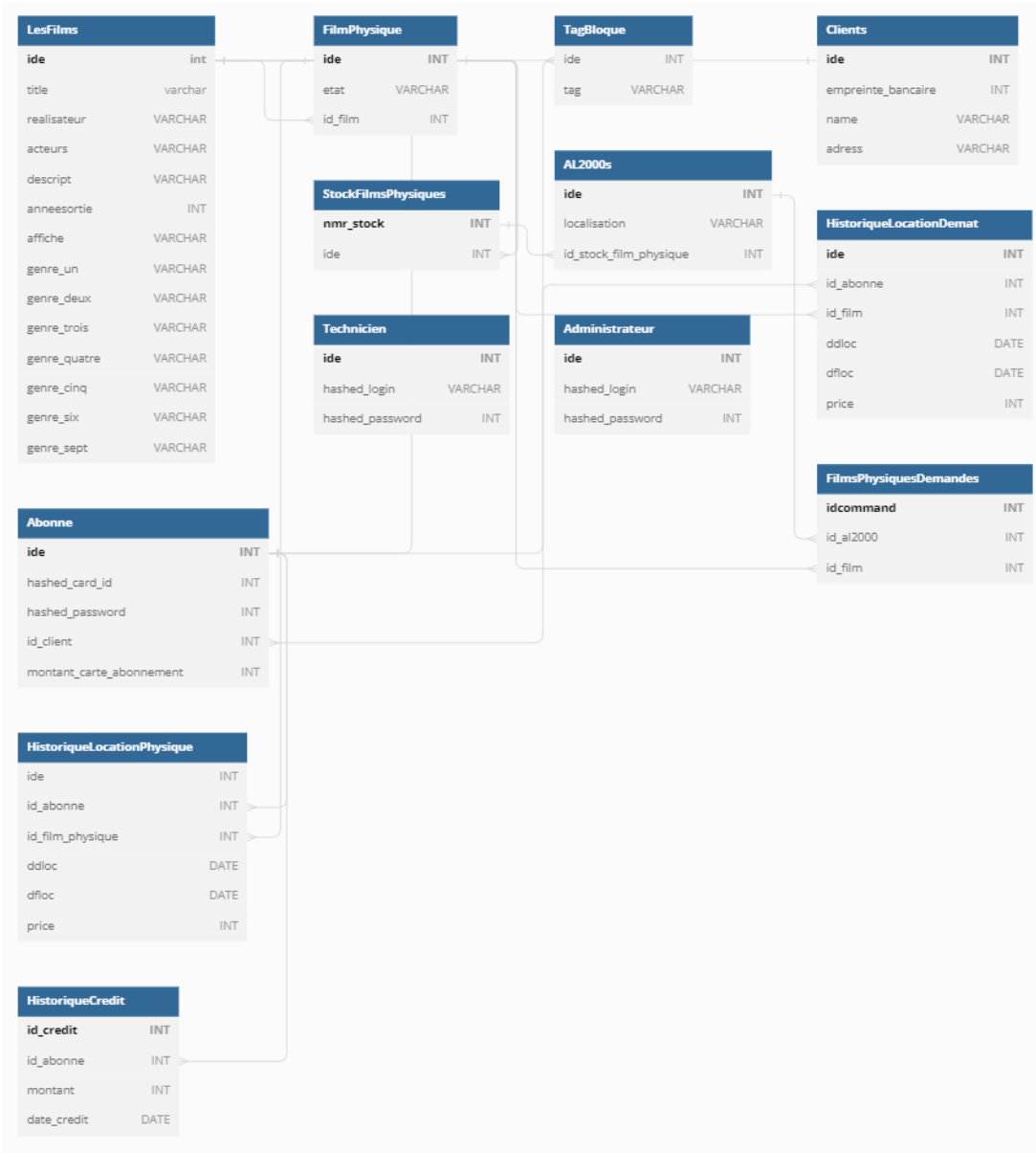


Figure 1: Schéma de la base de données